GitHub Copilot 워크플로 과정

Context Engineering ① Instructions

초안 → 다이어트 → 경로별 instructions → 전후 비교

1. 필요한 이유
2. 실습 1·2 다이어트
3. 세 종류 · 적용 범위
4. 실습 3·4 경로별·비교
5. 저장소 밖, 환경별 차이

01

Instructions가 필요한 이유

매번 같은 설명을 반복하지 않도록, 규칙을 파일로 저장해 두는 방법입니다.

Custom Instructions란

Copilot은 매 요청마다 이 프로젝트를 처음 보는 사람처럼 시작합니다.

Instructions는 매 요청에 자동으로 함께 전송되는 메모입니다.

- "시간은 UTC로", "테스트는 helpers 함수로" 같은 규칙을 매번 프롬프트에 쓰지 않기
- 파일로 저장 → 요청마다 자동 첨부
- 저장소 instructions 세 종류와 개인·조직 범위에 둘 수 있음

좋은 instructions vs 나쁜 instructions

좋은 instructions

- 이 프로젝트에만 있는 규칙
- 하지 말아야 할 것
- 작업 방식 (`python -m pytest -q` 전체 실행)
- 짧게 — 40줄 이내 권장

나쁜 instructions

- 일반론 (SOLID 원칙)
- 설치·실행 안내 (README 몫)
- 같은 말 반복
- 200줄짜리 설명서

Instructions는 고정 비용

모든 요청에 실리므로 길이가 곧 매 요청의 비용입니다.

40줄

권장 상한

200줄

흔한 설명서 길이

5배

매 요청마다 차이

- 일반론·README 내용은 매 요청 낭비
- "넣을까 말까" 고민되면 빼기

실습

instructions 초안 → 다이어트

일부러 긴 copilot-instructions.md 초안을 만든 뒤
프로젝트 고유 규칙만 남겨 40줄 이내로 줄이기

준비

- 실습 repo `main` 브랜치
- 새 세션: 하네스 `Copilot` · Interactive · Manual permissions
- 모델 GPT-6 Luna (다음 장)

실습 0

먼저 — 모델을 GPT-6 Luna로 고정

1. 입력창 아래 `Auto` → **GPT-6 Luna** 선택

2. 모델은 **새 대화를 시작할 때** 바꾸기

3. 실습이 끝나면 다시 `Auto` 로 복귀

✅ **입력창 아래 모델 표시가 GPT-6 Luna**

실습 1

instructions 초안 만들기 — 일부러 길게

1. 채팅 상단 + 로 새 대화, 첨부 없이 입력
2. 터미널 명령 승인 카드 → 내용 확인 후 **Allow in this Session**
3. 초안에 README 몫·일반론이 섞였는지 확인

프롬프트 / 명령

```
이 저장소의 구조, 코딩 규약, 테스트 방식을
분석해서 .github/copilot-instructions.md
초안을 작성해라. 모듈별 역할, 시간 처리 규칙,
새 알람 룰 추가 절차, 테스트 실행 방법,
하지 말아야 할 것을 포함해라.
```

✅ `.github/copilot-instructions.md` 생성 ✅ 약 80줄

실습 2

다이어트 — 40줄 이내로

1. 새 대화에서 `copilot-instructions` 첨부 (Files & Folders…)
2. 프롬프트 입력 후 삭제 대상과 최종본 확인
3. 사람이 검토 — 고유 규칙이 남았는지 확인

프롬프트 / 명령

```
첨부한 지침에서
(1) 모델이 이미 아는 일반론
(2) 이 프로젝트에만 해당하지 않는 문장
(3) 중복 을
골라 삭제 대상으로 표시해라.
남길 것은 프로젝트 고유 규약·금지·작업 방식뿐이다
.
40줄 이내 최종본을 만들어 파일에 반영해라.
```

✅ 40줄 이내 ✅ `datetime.now()` 금지 규칙 유지 ✅ `store` 직접 접근 금지 유지

02

저장소 instructions 세 종류

항상 / 경로가 맞을 때 / 부를 때만 — 무엇을 어느 파일에 둘지 고릅니다.

요청 1건에 instructions가 실리는 방식



② 경로별 instructions — applyTo

파일 맨 위 frontmatter에 적용 경로를 glob 패턴으로 지정합니다.

```
.github/instructions/tests.instructions.md

---
applyTo: "tests/**"
---
# 테스트 작성 규칙
- 시간은 `tests/helpers.py` 의 `T0` / `at()` / `tele()`
  만 사용.
```

1 **applyTo**

이 경로의 파일을 다룰 때만 실림

2 **위치**

`.github/instructions/` 아래

3 **excludeAgent**

`"code-review"` / `"cloud-agent"` 에서 제외

applyTo 패턴 예시

applyTo 예시	적용 대상
`"tests/**"`	`tests/` 아래 모든 파일
`"src/equip_monitor/api/**"`	API 라우터 폴더
`"**/*.py"`	저장소의 모든 `.py`
`"**/*.ts,**/*.tsx"`	쉼표로 여러 패턴
`"src/*.py"`	`src/` 바로 아래 `.py` 만 (하위 폴더 제외)

실습

경로별 instructions 2개 만들기

tests · api 폴더 전용 instructions를 만들고
applyTo 가 언제 실리는지 직접 확인하기

준비

- 모델 GPT-6 Luna, 실습마다 새 대화

실습 3-1

tests.instructions.md 만들기

1. 새 대화 → + 로 tests/helpers.py, tests/conftest.py 첨부

2. 프롬프트 입력 → 파일 생성 승인 카드 확인 후 **Allow in this Session**

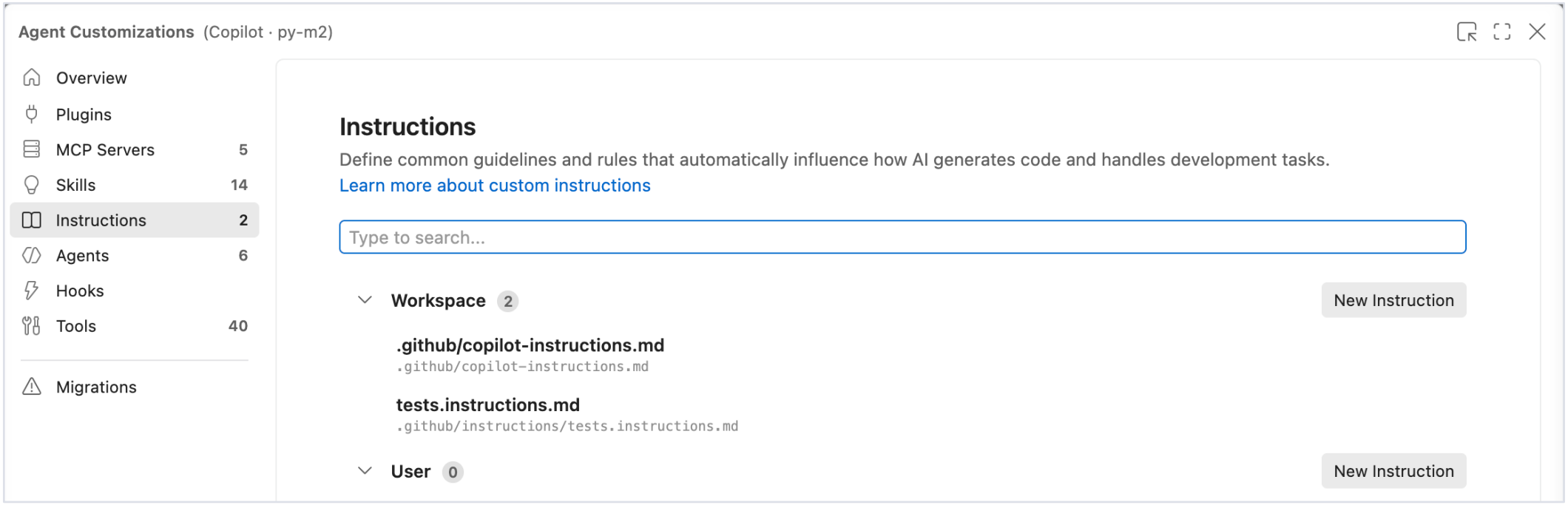
3. ⚙️ → Agent Customizations → **Instructions** 확인

프롬프트 / 명령

```
.github/instructions/tests.instructions.md 를 만들어라.
front matter applyTo: "tests/**".
내용: 파일·함수 명명 규칙,
시간은 helpers 의 T0/at/tele 만 사용,
경계 3종(미만/정확히/초과),
store 는 conftest 가 reset 하니 직접 초기화 금지,
네트워크·파일시스템 금지. 10줄 이내.
```

✅ 9줄 안팎, `applyTo: "tests/**"` ✅ Instructions 화면 Workspace 2개

Instructions 화면 — Workspace 2개



실습 3 이후의 Instructions 화면

1 **Workspace**

이 저장소의 instructions 파일

2 **User**

모든 프로젝트에 적용되는 내 instructions

3 **New Instruction**

여기서 새 instructions 파일 만들기

실습 3-2

api.instructions.md 만들기

1. 새 대화 → + 로 api/telemetry.py, models.py 첨부
2. 프롬프트 입력 → 결과 확인
3. 규칙이 한 줄에 몰리면 "규칙마다 한 줄" 요청

프롬프트 / 명령

```
.github/instructions/api.instructions.md 를 만들어라.
front matter applyTo:
"src/equip_monitor/api/**".
내용: 라우터는 도메인별 파일 하나와
APIRouter(prefix, tags) 로 만들고
app.py 의 create_app 에 등록,
경로·쿼리 datetime 은 models.ensure_utc 사용,
없는 리소스는 404·입력 오류는 422,
응답 타입은 models.py 모델 재사용,
계산 로직은 report/·alarms/ 로 위임. 6줄 이내.
```

실습 3-3

applyTo 가 언제 실리는지 확인

1. **F1 → Developer: Reload Window** 먼저 실행
2. 새 대화에서 ① 계획만 요청
3. 새 대화에서 ② 실제 수정 요청
4. 끝에 "적용된 지침 파일 경로" 질문 붙여 비교
5. ② 변경은 확인 후 `git checkout -- src tests`

요청 ① / ②

```
① api/devices.py 에
   GET /devices/{device_id}/alarms 를
   추가하려면 어떻게 할지 계획만 5줄로

② api/devices.py 에
   GET /devices/{device_id}/alarms
   엔드포인트를 추가해라
```

③ 부를 때만 — 프롬프트 파일에서 Skill로

프롬프트 파일 (예전)

- `.github/prompts/*.prompt.md`
- Copilot 하네스는 읽지 않음
- Docs: preview · IDE 전용

Skill

- `.github/skills/` 아래 `<이름>/SKILL.md`
- VS Code·CLI·앱·클라우드·리뷰

입력창에 `/이름` 으로 실행 — 예: `/explain-flow` 는 `disable-model-invocation: true` 로 부를 때만 실림

Instructions 끄기와 효과 비교

끄기

파일 이름 바꾸기

가장 확실한 방법 — 설정 항목은 경로별 instructions 미대상

효과

예시가 없는 새 작업

instructions 없으면 현재 코드에, 있으면 의도에 맞춤

실습

instructions 있을 때 vs 없을 때

같은 프롬프트를 instructions 끈 상태 · 켠 상태로 보내
결과를 비교하기

준비

- 각 요청은 **새 대화**에서
- before 결과는 비교 후 버리기

실습 4-0

instructions 끄고 켜기 명령

1. 끄기 → **Reload Window** → (a)·(b) before, 매번 버리기
2. 켜기 → **Reload Window** → (a)·(b) after
3. before 는 기록해 두고 after 와 비교
4. 켜기를 잊지 않기 — Workspace 3개 확인

명령 (터미널)

```
# 끄기
mv .github/copilot-instructions.md
.github/copilot-instructions_off.md
mv .github/instructions .github/instructions_off

# 켜기
mv .github/copilot-instructions_off.md
.github/copilot-instructions.md
mv .github/instructions_off .github/instructions

# before 결과 버리기
git checkout -- tests/
git clean -fd tests/
```

실습 4-1

(a) 기존 테스트에 경계값 추가

1. 끄고 **Reload Window** → 새 대화에서 (a) → before 기록
2. before 결과 버리기 → (b) before 까지 끈 상태로
3. 켜고 **Reload Window** → 같은 프롬프트 → after 비교

프롬프트 / 명령

```
temp-critical 룰의 경계 테스트
(94.9 / 95.0 / 95.1)를 추가해라.
테스트 실행은 하지 마.
```

✅ **before·after 거의 같음 (둘 다 `tele()` 사용)**

실습 4-2

(b) 예시가 없는 새 테스트 파일

1. 매번 **Reload Window** 후 before / after 비교
2. before: 현재 구현(로컬 시간대)에 맞춤
3. after: `Asia/Seoul` 기준 → UTC면 실패
4. 적용 확인 문장 붙여 보기 → 커밋

프롬프트 (b) / 적용 확인 / 커밋

```
새 테스트 파일 tests/test_day_range.py 를
만들어 report/uptime.py 의 day_range 함수를
검증해라. 테스트 실행은 하지 마.

마지막 줄에 이번 작업에 적용된
지침 파일 경로를 적어라.

git add .github
git commit -m "docs: copilot, tests, api
instructions"
git push
```

03

저장소 밖 · 환경별 차이

개인 · 조직 instructions와 우선순위, 그리고 VS Code · CLI · 앱의 차이입니다.

세 가지 범위 비교

범위	누가 만드나	어디에	적용되는 곳	우선순위
개인	나	GitHub.com Chat **Personal instructions** / CLI `~/.copilot/`	나의 모든 저장소 (Chat, CLI)	**1**
저장소	팀 (repo에 커밋)	`.github/` 의 `copilot-instructions.md` · `instructions/`	저장소 사용자 전체	2
조직	조직 Owner	Org Settings → Copilot → Custom instructions	조직 구성원 전체	3

조직(Organization) instructions

엔터프라이즈 환경에서 조직 Owner가 설정합니다.

1. **필요 조건**

   Copilot **Business** / **Enterprise** 플랜, 조직 Owner 권한

2. **적용 범위**

   GitHub.com Copilot Chat · code review · cloud agent 만

3. **적용 안 되는 곳**

   VS Code · Copilot CLI · GitHub Copilot 앱 등 IDE·로컬 도구

4. **넣을 것**

   보안·컴플라이언스·언어처럼 모든 repo 공통 규칙만

VS Code · Copilot CLI · GitHub Copilot 앱

	VS Code	Copilot CLI	GitHub Copilot 앱
저장소 instructions	`.github/` 아래 파일	같음 (+ 현재·중간 폴더 탐색)	같음 (CLI 기반)
개인 instructions	Instructions → **User**	`~/.copilot/copilot-instructions.md`	사이드바 **Customize**
Instructions 목록	Agent Customizations	`/instructions` (켜고 끄기)	사이드바 **Customize**
변경 반영 시점	저장 즉시	**새 세션부터** (`/new`)	새 세션부터
Skill 실행	`/이름`	`/이름`, `/skills`	`/이름`, `/skills`

`.github/` 아래 instructions·Skill은 세 환경이 같은 위치에서 읽음

핵심 요약

1. Instructions = 매 요청에 자동으로 붙는 메모 — 40줄 이내, 프로젝트 고유 규칙만
2. 저장소 instructions 세 종류: 항상 / 경로별(`applyTo`) / 부를 때만(Skill `/`이름)
3. `applyTo` 는 그 경로의 파일을 실제로 다룰 때 실림
4. 범위: 개인 > 저장소 > 조직 우선, 모두 함께 전달
5. 효과는 예시가 없는 새 작업에서 — 없으면 현재 코드, 있으면 의도
6. 이 실습은 GPT-6 Luna로 — 끝나면 Auto로 복귀